

APP DEVELOPMENT PROJECT REPORT

1. Student Details

Name: Abhishek Tiwari

Roll Number: 24F2006100

Email: 24f2006100@ds.study.iitm.ac.in

2. Project Details

Placement Portal Application

Problem Statement:

To build a Placement Portal web application that allows Admin, Company, and students to interact with the system based on their roles.

Approach:

The App was developed using Flask as Backend Framework, while SQLAlchemy ORM was used to manage the SQLite database. The frontend was built using Jinja2 templates, HTML, CSS, and Bootstrap, allowing dynamic rendering of data and responsive user interfaces.

3. AI/LLM Declaration

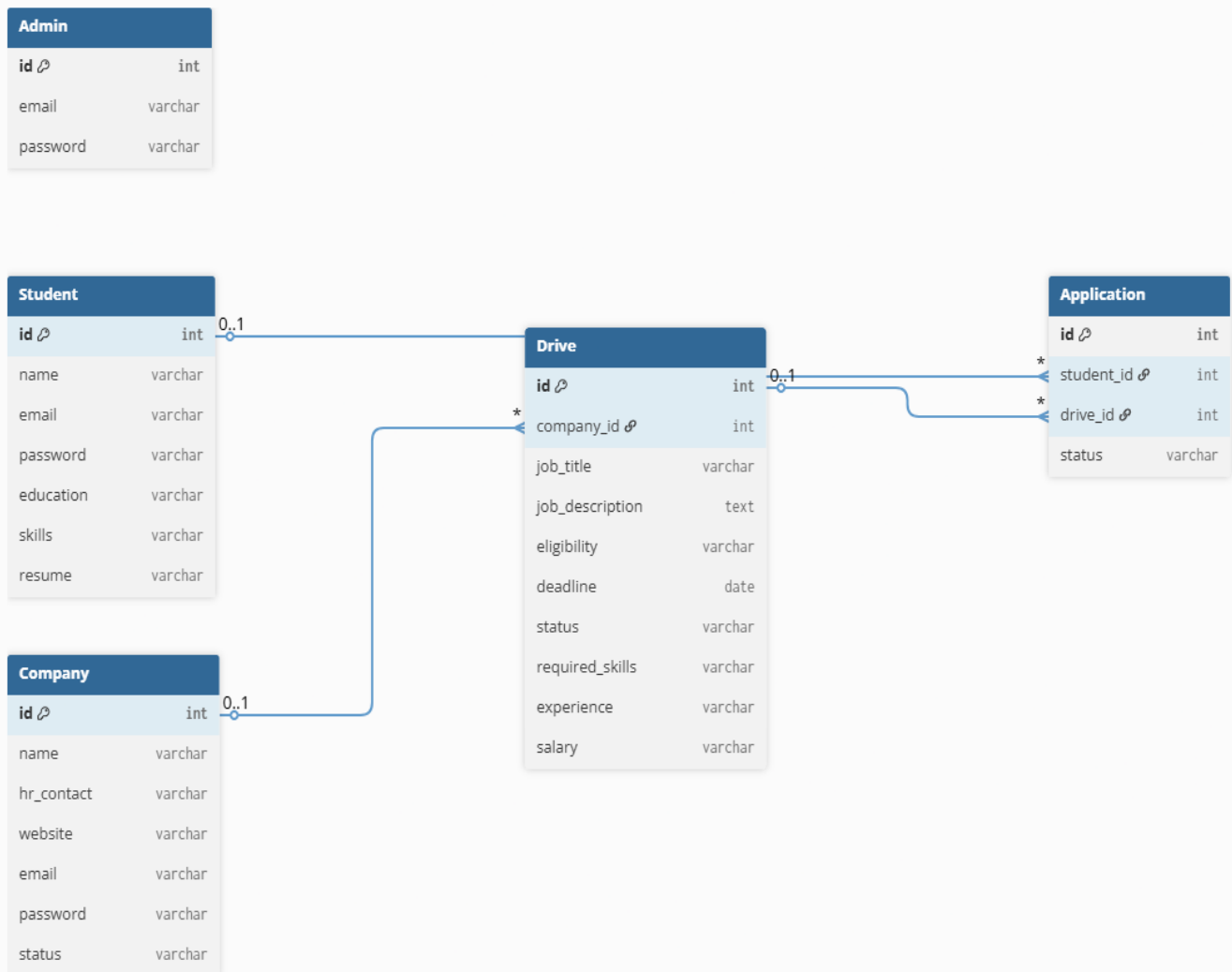
I used ChatGPT (GPT-5) to assist me in this project. The extent of AI/LLM usage is around 10–15%, limited to code suggestions and documentation formatting.

All final implementation logic, debugging, and integration were done manually.

4. Technologies and Frameworks Used

Technology	Purpose
Flask	Backend web framework used to build the application and handle routing
SQLAlchemy	ORM used for database models and relationships
SQLite	Local database used to store application data
Jinja2	Template engine used to generate dynamic HTML pages
HTML	Structure of web pages
CSS	Basic styling for user interface
Bootstrap	Responsive frontend design and UI components
Flask-Login	User authentication and session management
Werkzeug	Password hashing and security

5. Database Schema / ER Diagram



Tables:

1. Admin — stores administrator login details (*id, email, password*)
2. Student — stores student profile information (*id, name, email, password, education, skills, resume*)
3. Company — stores company profile details (*id, name, hr_contact, website, email, password, status*)
4. Drive — stores placement drive information (*id, company_id, job_title, job_description, eligibility, deadline, status, required_skills, experience, salary*)
5. Application — stores student applications for drives (*id, student_id, drive_id, status*)

Relationships:

- One-to-Many → Company → Drive
A company can create multiple placement drives.
 - One-to-Many → Drive → Application
A placement drive can receive applications from multiple students.
 - One-to-Many → Student → Application
A student can apply to multiple placement drives.
-

6. API Resource Endpoints

This project primarily uses Flask routes for server-side operations.

Endpoint	Method	Description
/login	POST	Authenticates user login
/register_student	POST	Registers a new student
/register_company	POST	Registers a new company
/student_dashboard	GET	Displays student dashboard
/company_dashboard	GET	Displays company dashboard
/admin_dashboard	GET	Displays admin dashboard
/create_drive	POST	Company creates a placement drive
/apply_drive	POST	Student applies for a drive
/update_status	GET	Company updates application status

7. Architecture and Features

The project structure is organized as follows:

- **app.py** — main Flask application containing routes and application logic
- **models.py** — database models defined using SQLAlchemy
- **config.py** — configuration settings for the Flask application
- **templates/** — HTML templates rendered using Jinja2
- **static/** — static files such as uploaded resumes

Implemented Features:

- **Role-based authentication system** for Admin, Company, and Student users
 - **Admin dashboard** for managing companies, students, placement drives, and applications
 - **Company dashboard** for creating and managing placement drives and reviewing student applications
 - **Student dashboard** for viewing approved drives, searching companies, and applying for jobs
 - **Application status tracking** allowing companies to update student selection status
 - **Resume upload and profile management** for students
 - **Search functionality** for companies and students
 - **Prevention of duplicate applications** for the same drive
-

8. Video Presentation**Drive Link:**

<https://drive.google.com/file/d/1XSuzBG9sB4nb3YaAecsfa9OQAII9iBT4/view?usp=sharing>